CA documentation

CA:	No 2.
Title:	Alternate Controller
Name:	Matthew Seymour
Date submitted:	02/11/2026

Description

This project is an alternate controller for the browser driving game Slow Roads [1]. Instead of using the keyboard to steer, the player steers by holding out their arm and pointing in the direction they want to steer. A HC-SR04 ultrasonic sensor measures the distance from itself to the player's arm. The distance is then converted into a steering direction by the code on the Arduino Leonardo.

Three buttons handle the remaining controls by emulating keyboard presses: W to accelerate, S to reverse/brake, and R to reset the car onto the road.

A fourth button toggles a safety that disables all other controls. The safety state is shown by an RGB LED: red means the safety is on and the controls are disabled, and green means the safety is off and the controls are enabled.

How to use the project

Place the alternate controller on the table in front of you, slightly to the right, where a computer mouse would be. Ensure that the ultrasonic sensor is aiming to the left. Plug the Leonardo into the computer and open the Slow Roads game in a browser (https://slowroads.io/).

The safety is on by default, so the light from the RGB LED will be red. When you are ready to play the game, press the button beside the RGB LED to toggle the safety. Hold your arm in front of the ultrasonic sensor to steer the car. The closer your arm is to the sensor, the more the car will turn right. The further away your arm is, the more the car will turn left.

To accelerate, brake, or reset the car back onto the road, press and hold the corresponding button.

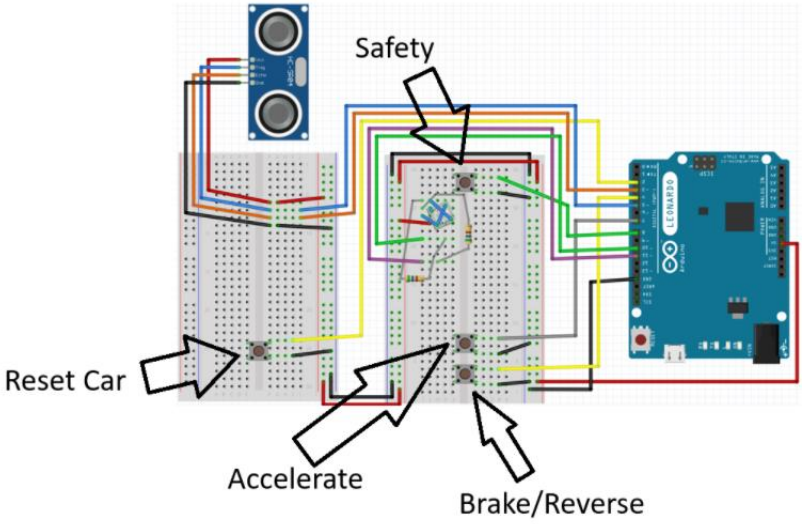


Figure 1 Controls Diagram

To turn on/off the light from the RGB LED that shows the safety state, hold down the safety button for one second. Doing this does not change the safety state, and the safety button still works while the light is off.

Hardware

Item	Quantity	Notes
Arduino Leonardo	1	
HC-SR04 Ultrasonic Sensor	1	To measure the distance from the hand/arm to the sensor, and steer the car accordingly.
Piranha RGB LED (Common Anode)	1	To show if the safety is on by showing red or green. Red shows that the board is disconnected, and that the safety is on. Green shows that the board is connected, and that the safety is off.
560Ω Resistor	2	For the red and green pin of the piranha RGB LED. A third resistor is not needed as the only two colours I'm using are red and green, therefore there is no need from the blue pin at all.
Breadboard	2	The second breadboard is for extra space and user comfortability.
Button	4	Three of the buttons control keys on the keyboard. 'W' for accelerating the car forward, 'S' for reversing/braking, and 'R' for resetting the car back on the road. The fourth button is a toggle that enables/disables the rest of the controls.

Electronics

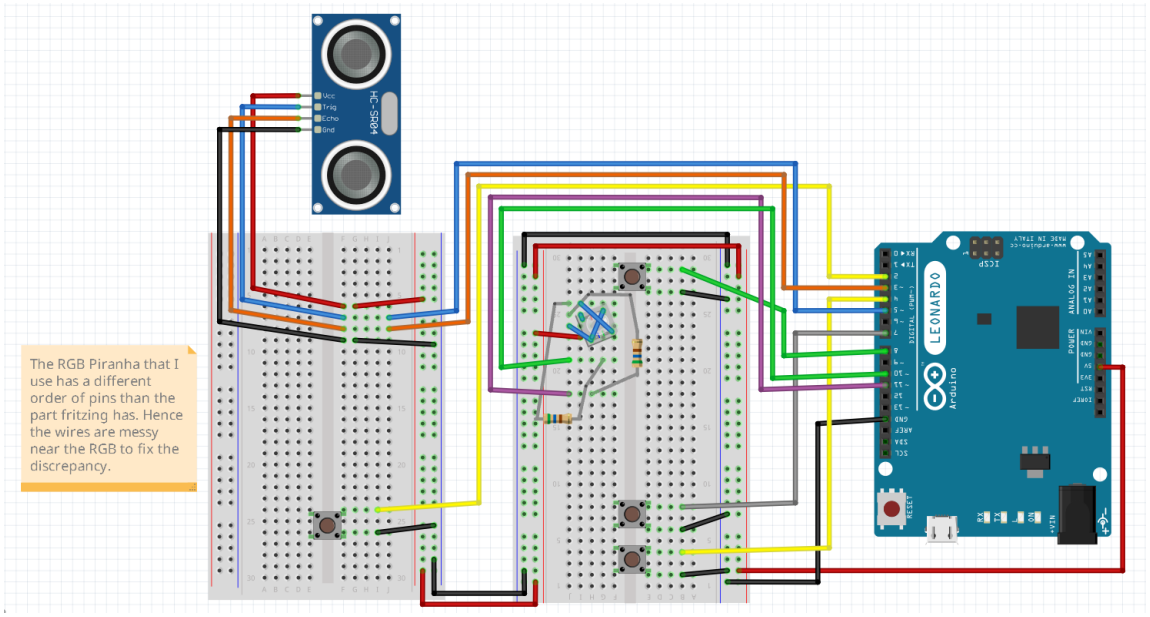


Figure 2: Fritzing diagram

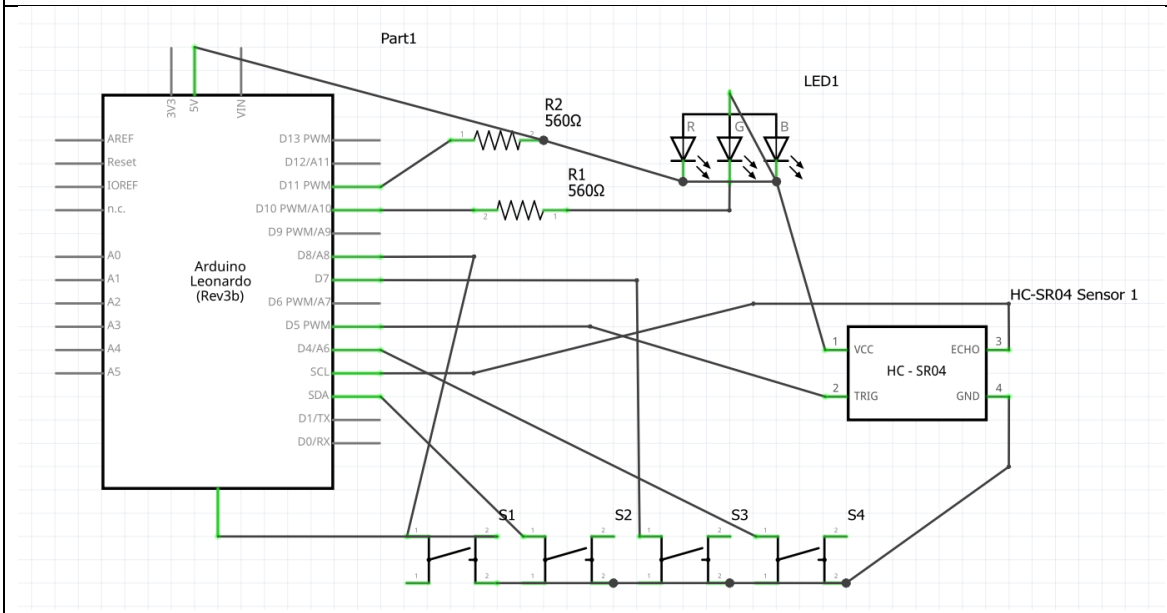


Figure 3: Schematic

Pin Connections

Arduino Leonardo Pin	Device	Device Pin
2	Button (R Key)	One leg of button

Arduino Leonardo Pin	Device	Device Pin
3	HC-SR04 Ultrasonic Sensor	Echo
4	Button (S Key)	One leg of button
5	HC-SR04 Ultrasonic Sensor	Trig
7	Button (W Key)	One leg of button
8	Button (Safety)	One leg of button
10	Piranha RGB LED (Common Anode)	Green
11	Piranha RGB LED (Common Anode)	Red

Project Structure

Hardware

The code is in a single Arduino sketch. As shown in Figure 4 it uses the HID-Project library [2], which allows the Leonardo to appear as both a keyboard and a gamepad over USB. The gamepad is used for steering because its joystick axis gives a range of values, which suits a steering wheel much better than the binary on/off of a keypress.

Each button used is an instance of the Button class, as shown in Figure 4. The class allows multiple buttons to share the same methods, which avoids the need to repeatedly write the same code for each button. Each instance stores its own set of variables, known as data members, which are declared in the class's public section. The constructor allows selected data members to be initialized with values supplied when creating each instance. This allows each instance to have different values for their data members, causing the class's methods to behave differently for each instance, despite the code remaining the same.

```ino
// <HID-Project.h> is a library that has gamepad functionality, which was used for the joystick.
// It also has Keyboard functionality so I removed the initial '#include <Keyboard.h>' as
// having both would result in an error.
#include <HID-Project.h>

class Button {
  public:
    int pin;
    char letter;
    int value = HIGH;
    int oldValue = HIGH;
    unsigned long lastDebounceTime = 0;
    int reading;

    Button(int p, char l) {
      pin = p;
      letter = l;
    }
    void setPinMode() {
      pinMode(pin, INPUT_PULLUP);
    }
    void getReading() {
      reading = digitalRead(pin);
    }
    void debounce(unsigned long debounceDelay) {
      if (reading != oldValue) {
        lastDebounceTime = millis();
      }
      if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading != value) {
          value = reading;
        }
      }
      oldValue = reading;
    }
    void buttonControl() {
      if (value == LOW) {
        Keyboard.press(letter);
      }
      else {
        Keyboard.release(letter);
      }
    }
};
```

Figure 4 Button class

As shown in Figure 5, the button instances are created, each with their own values (pin and letter) passed as parameters to the constructor. The safety button uses '\0', which represents a null character, because it uses separate logic in the code, rather than sending a keypress to the keyboard.

```
45
46  Button wButton(7, 'w');
47  Button sButton(4, 's');
48  Button rButton(2, 'r');
49  Button safetyButton(8, '\0');
50
```

Figure 5 Create button instances

As shown in Figure 6, the setup function sets the pin modes for all the pins used. Each button instance does this by calling the setPinMode() method, which is defined in the Button class shown in Figure 4. The buttons use INPUT_PULLUP, which makes use of the Leonardo's built in resistors. This is also why a button reads LOW when pressed. The other pins used are for the ultrasonic sensor and the RGB LED. The setup function then initializes the keyboard and gamepad functionality.

```
73  void setup() {
74    Serial.begin(9600);
75    safetyButton.setPinMode();
76    wButton.setPinMode();
77    sButton.setPinMode();
78    rButton.setPinMode();
79    pinMode(trigPin, OUTPUT);
80    pinMode(echoPin, INPUT);
81    pinMode(redLEDPin, OUTPUT);
82    pinMode(greenLEDPin, OUTPUT);
83    Keyboard.begin();
84    Gamepad.begin();
85  }
```

Figure 6 Setup function

As shown in Figure 7, millis() is used to limit how often the following code runs. Without this limiter, the code would run more often than necessary. The code only runs while the safety is off. It triggers the ultrasonic sensor and calculates the distance to the detected object. This reading is only accepted if it is within the required range, or within 10% of the range's width beyond either end. Readings further outside than this are discarded as errors. Accepted readings that fall just outside the range are clamped to the limits.

Next, a rolling median filter is applied to prevent inaccurate readings from affecting the result. The rolling median filter keeps track of the 5 most recent readings and calculates their median value using the median() function, as shown in Figure 8. The median is used instead of the average

because the average would be pulled towards outliers, whereas the median is not. The median value is then used as the filtered distance. After each new reading, the array is updated to contain the 5 most recent readings. This is more efficient than taking 5 new readings each time the median is calculated. Finally, the filtered value is mapped from the minimum and maximum distance range to the minimum and maximum values accepted by the joystick.

```
87  void loop() {
88
89    // Reads and filters the ultrasonic sensor distance, then maps the value to the joystick's range.
90    if (millis() - lastMeasurementTime > debounceDelay && safetyIsOn == false) {
91      digitalWrite(trigPin, LOW);
92      delayMicroseconds(2);
93      digitalWrite(trigPin, HIGH);
94      delayMicroseconds(10);
95      digitalWrite(trigPin, LOW);
96      duration = pulseIn(echoPin, HIGH);
97      distanceReading = (duration * .0343) / 2;
98
99      // Only accepts new readings that are within the range of maximum to minimum ± 10%.
100     if (distanceReading < (maximumDistance + tenPercent) && distanceReading > minimumDistance - tenPercent) {
101       distance = distanceReading;
102       // Set/Enforce minimum and maximum distance.
103       if (distance > maximumDistance) {
104         distance = maximumDistance;
105       }
106       if (distance < minimumDistance) {
107         distance = minimumDistance;
108       }
109
110       // Using a rolling median filter to shield against bad distance readings.
111       // Rolling the values of the recent readings back one, then adds the new distance value.
112       for (int i = 0; i < 4; i++) {
113         recentDistanceReadings[i] = recentDistanceReadings[i + 1];
114       }
115       recentDistanceReadings[4] = distance;
116       // printArrayValues(recentDistanceReadings, 5); // For debugging.
117
118       // Uses median function
119       medianValue = median(recentDistanceReadings, 5);
120
121       // The range of values that the joystick reads is 32767 to -32768.
122       mappedJoystickDistance = map(medianValue, minimumDistance, maximumDistance, 32767, -32768);
123     }
124     lastMeasurementTime = millis();
125   }
```

Figure 7 Ultrasonic sensor reading and filtering

```
193  //Function to get the median value of the elements in an array.
194  float median(const float arr[], int size) {
195    float arrayCopy[size];  // Variable-length array (compiler-dependent).
196
197    // Makes a copy of the array.
198    for (int i = 0; i < size; i++) {
199      arrayCopy[i] = arr[i];
200    }
201    // Sorts the copy.
202    for (int i = 0; i < size - 1; i++) {
203      for (int j = 0; j < size - i - 1; j++) {
204        if (arrayCopy[j] > arrayCopy[j + 1]) {
205          float t = arrayCopy[j];
206          arrayCopy[j] = arrayCopy[j + 1];
207          arrayCopy[j + 1] = t;
208        }
209      }
210    }
211
212    if (size % 2 == 1) {
213      return arrayCopy[size / 2];
214    }
215    return (arrayCopy[size / 2 - 1] + arrayCopy[size / 2]) / 2;
216  }
```

Figure 8 Median function

As shown in Figure 9, the button instances call the debounce() method, shown in Figure 4, to stop electrical fluctuations from being read as extra button presses. Because these fluctuations are extremely short, the code only accepts a reading if it stays the same for long enough. In this case, the code waits to see if the input is the same for longer than the debounceDelay variable, which is 50 milliseconds.

The safety button is handled separately because it also detects long presses. A short press of the safety button toggles the safety, while a long press toggles the LED. If the button is held for more than a second, the long press logic triggers immediately without waiting for the button to be released. For a short press, the code waits for the button to be released before toggling the safety, provided a long press hasn't already been detected. This ensures a long press doesn't also toggle the safety.

```
133  // Debounces the buttons. The safety button is handled separately because
134  // it also detects long presses to enable/disable the LED.
135  wButton.debounce(debounceDelay);
136  sButton.debounce(debounceDelay);
137  rButton.debounce(debounceDelay);
138  if (safetyButton.reading != safetyButton.oldValue) {
139    safetyButton.lastDebounceTime = millis();
140  }
141  if ((millis() - safetyButton.lastDebounceTime) > debounceDelay) {
142    if (safetyButton.reading != safetyButton.value) {
143      safetyButton.value = safetyButton.reading;
144      if (safetyButton.value == LOW) {
145        safetyPressStartTime = millis();
146        safetyLongPressed = false;
147      }
148      else if (!safetyLongPressed) {
149        safetyIsOn = !safetyIsOn;
150        Serial.println(safetyIsOn ? "safety IS on" : "safety NOT on");
151      }
152    }
153  }
154  if (safetyButton.value == LOW && safetyLongPressed == false && (millis() - safetyPressStartTime) > holdTime) {
155    safetyLEDIsOn = !safetyLEDIsOn;
156    safetyLongPressed = true;
157    Serial.println(safetyLEDIsOn ? "safety LED IS on" : "safety LED NOT on");
158  }
159  safetyButton.oldValue = safetyButton.reading;
```

Figure 9 Debounce buttons

Interface

The code shown in Figure 10 determines what inputs are actually sent to the computer. If the safety is on, the controller inputs should be disabled, so the code releases all keys and sends a neutral joystick input to the computer. If the safety is off, the joystick value is sent to the computer, and each button instance calls the buttonControl() method shown in Figure 4, which checks if the button is currently pressed, and if so, sends that keypress to the computer, otherwise it releases its respective key.

The final piece of code controls the LED, which can be enabled or disabled. If it's disabled, then the light is simply turned off. If it's enabled, it shows a red light if the safety is on, and a green light if the safety is off.

```
161   // Button and wheel controls
162   if (safetyIsOn == false) {
163     wButton.buttonControl();
164     sButton.buttonControl();
165     rButton.buttonControl();
166     Gamepad.xAxis(mappedJoystickDistance);
167     Gamepad.write();
168   }
169   else {
170     Keyboard.releaseAll();
171     Gamepad.xAxis(0);
172     Gamepad.write();
173   }
174   // LED controls. Changing the colour of the light to red or green depending on the safetyButton state.
175   // And turning off the LED entirely if it's turned off.
176   // Because the RGB LED has a common anode, 255 = OFF and 0 = ON.
177   if (safetyLEDIsOn) {
178     if (safetyIsOn) {
179       analogWrite(redLEDPin, 0);
180       analogWrite(greenLEDPin, 255);
181     }
182     else {
183       analogWrite(redLEDPin, 255);
184       analogWrite(greenLEDPin, 0);
185     }
186   }
187   else {
188     analogWrite(redLEDPin, 255);
189     analogWrite(greenLEDPin, 255);
190   }
191 }
```

Figure 10 Button, wheel, and LED controls

User Testing

Two users tested the controller informally as part of a guerrilla testing approach. Each user played Slow Roads with the controller and gave feedback on their experience.

Results

Positives

- Both users felt that the controls were very intuitive.
- Neither user reported discomfort when using the buttons.
- First impression: "Unique and cool, but slightly uncomfortable".
- The controller functioned as intended.

Negatives

- The controller was slightly uncomfortable to hold. One user expected they would experience fatigue from prolonged use of the sensor controls.
- The sensor and buttons are fixed in place and can't be positioned separately from each other. One user said "I don't like that the sensor is attached to the controls, because it

makes it more clunky, and forces the button controls to be on the table. I want to be able to hold the controller in my hand".

- One user wanted the controller to be wireless, but said that it would still likely be uncomfortable to hold.
- Users can move beyond the sensor's range without realising, except on the near side, where the sensor itself gets in the way. Going past the far limit means they must overcorrect when turning back the other way.
- Communicating which button controlled each action was difficult because the buttons were not labelled.
- One user wanted the controller to vibrate when they pressed a button.

Analysis

The controller worked as intended, and users found the controls intuitive and easy to grasp. However, testing highlighted several shortcomings, namely the controller's poor ergonomics and the lack of feedback to the user.

A major problem was that the controller was uncomfortable to use. This could be improved by mounting the system in a case and changing the button positions. However, the general layout should remain the same, as users found it intuitive. A case would also hide the exposed wires.

The controller gives users no way of knowing where the limits of the ultrasonic sensor are, making it easy to move their arm beyond its range without realising. This creates a need to overcorrect when turning back. Adding physical markers to show the sensor's maximum and minimum limits would help users stay within the sensor's range.

The buttons provide limited feedback when pressed. Adding a simple buzzer or vibration would give users clearer confirmation that their input has been registered. Additionally, the buttons are identical and unlabelled, making it difficult to communicate which button controls each action. Giving each button is own label or symbol would make them easier to identify.

Since the ultrasonic sensor and the buttons are attached to the same unit, users cannot position them independently. Separating the two components would allow users to position them to their preference. However, this would likely require the controller to be wireless, adding complexity and requiring a separate power source, such as a battery.

Reflection

I learnt how to work with the breadboard and Arduino Leonardo much more confidently. I now have a better understanding of what different components require and how to set them up, without needing to follow a tutorial. Building the project helped solidify my understanding, as I had to apply what I learned from the tutorials to my own project rather than following them directly. I also learnt how to use classes, which allowed me to avoid writing the same code multiple times, as well as improve the readability of the code.

I encountered difficulties when trying to get the ultrasonic sensor to control the steering of the car. This was mainly because I didn't realise that Slow Roads accepted controller input, so I initially tried to translate the sensor's readings into keyboard input. Since keyboard inputs are binary, it was very difficult to control varying degrees of steering. When I discovered that I could use a joystick as the input, I changed my code to use it. It was much easier to implement and produced a better result.

I would develop this project further by carrying out some of the suggestions in the user testing analysis. This would include adding a case to improve ergonomics and to hide the wires. I would also add markings to indicate the ultrasonic sensor's range, as well as adding vibration feedback when buttons are pressed and symbols to identify each button. A stretch goal would be to make the controller wireless and separate the ultrasonic sensor from the button controls.

References

[1] Driving game: https://slowroads.io/
Accessed 29th October 2026

[2] HID-Project library by NicoHood, version 2.8.4: https://github.com/NicoHood/HID
Accessed 31st October 2026